

实验 4 定位实验

一、实验目的

- 1、了解机器人绝对定位与相对定位的概念、特点及典型技术实现。对比分析不同定位方式的适用场景、定位精度与稳定性。
- 2、掌握基于粒子滤波的蒙特卡洛定位方法的基本原理和定位流程。通过调整功能包相关参数，分析粒子数量、模型参数对定位效果的影响。。
- 3、掌握 ROS 导航栈中核心坐标系（map、odom、base_footprint）的含义与变换机制，建立对机器人在移动过程中的空间状态估计与坐标转换体系（TF 树）的系统认知。

二、实验环境与器材

- 1、实验硬件设备
计算机、移动机器人导航平台
- 2、实验软件配置
Ubuntu20.04, ros-noetic
- 3、实验场景
教学实验场地

三、实验原理

1. 机器人定位

定位技术是移动机器人自主导航体系中的核心模块，旨在解决“我在哪里”这一根本问题，核心任务是通过传感器感知与数据解算，实时、准确地确定机器人在工作环境中的位置与朝向信息（统称为位姿）。根据定位原理与参考坐标系的不同，机器人定位主要分为相对定位与绝对定位两大类。

1) 相对定位也叫作位姿跟踪。以机器人自身初始位置为参考基准，采用相邻时刻传感器信息对机器人位姿进行跟踪估计。相对定位方法主要有里程计法和惯性导航法等。

里程计法（论速计）：基于安装在驱动轮上的光电编码器，实时采集车轮的转速与转角脉冲信息，实现位姿跟踪。其核心原理是航位推算，即在假定初始位姿已知的条件下，结合机器人的运动学模型，将车轮的角位移映射为底盘的线速度与角速度，并通过时间积分持续迭代更新机器人在局部坐标系下的实时位姿。

惯性导航法：以惯性测量单元（IMU）为核心传感器。机器人从已知初始状态出发，利用内部集成的陀螺仪和加速度计，分别实时感知机体坐标系下的三轴角速度与线加速度。通过对角速度进行一阶积分解算姿态角（航向），并对线加速度进行二次积分推算位移与速度，从而实现不依赖外部环境的自主位姿估计。

相对定位的优势在于响应速度快、数据连续性以及短距离精度高，且不依赖外部环境特征。然而，其致命缺陷在于航位推算是个累加过程，误差会随着时间的推移和运动距离的增加不断累积（即漂移现象），导致长时间运行后定位结果严重偏离真实值。

2) 绝对定位又称为全局定位，以环境固有特征、外部基准或预设地图为参考，通过传感器直接或间接获取机器人在全局坐标系下的绝对位姿，不存在累积误差，能够实现全局初始化与位姿修正，是解决定位漂移的关键技术。常见的绝对定位方法有地图匹配、信标定位、全球卫星导航系统（GPS）和超宽带（UWB）基站等

地图匹配定位：利用传感器感知环境信息，然后将当前数据与预先构建的栅格地图或特征地图进行匹配，计算出机器人在全局坐标系中最优位姿。

信标定位：利用环境中固有的自然路标或在环境中部署人工标记，通过识别标记坐标和三角测量原理实现精准绝对定位。

GPS 定位：依托卫星信号获取全局地理坐标，主要用于室外机器人的导航与定位，最常见的应用场景就是无人驾驶汽车，通过 RTK-GPS 技术，为无人驾驶汽车提供厘米级的高精度定位结果。

UWB 定位，通过基站与标签通信实现室内高精度全局定位。

绝对定位的优势在于其定位误差是有界的，不会随时间累积，能够为机器人提供全局一致的坐标参考。其缺陷在于容易受到环境干扰（如 GPS 在室内无信号、视觉受光照影响），且部分信标系统的前期部署和后期维护成本较高，至于地图匹配定位，高精度通常伴随着高昂的计算复杂度。

在实际应用中，为了实现高效可靠的定位效果，普遍采用多传感器融合定位技术，将相对定位的短期平滑性与绝对定位的长期稳定性等优势相结合，显著提升了定位系统的实用性与鲁棒性。

	方法举例	优点	缺点
相对定位	航迹推演，车道跟踪	简单，易用	不能提供绝对位姿，无法回答导航中“我要去哪儿？”和“我怎样到达那里？”的问题；累积误差
地图匹配	蒙特卡罗，NDT	全局定位，成本低	对算力有要求，定位精度取决于算法性能
信号定位	UWB，蓝牙，GPS	全局定位 无高算力需求	不稳定，信号易受遮挡或干扰；前期部署和后期维护成本高
轨道定位	磁导轨，二维码	全局定位 定位精度极高	机器人只能按照预设的轨迹运动，无法实现自由导航

2. 蒙特卡洛定位

蒙特卡洛定位（Monte Carlo Localization, MCL）是一种基于粒子滤波的概率定位算法，其核心思想是通过一组带权重的随机样本（即“粒子”）来近似表示机器人位姿的后验概率分布，从而在存在传感器噪声和运动不确定性的情况下，实现对机器人位置的鲁棒估计。自适应蒙特卡洛定位（Adaptive Monte Carlo Localization, AMCL）是 MCL 的改进版本，通过引入自适应机制解决了传统 MCL 的两大核心局限：固定粒子数导致的效率问题和绑架恢复能力不足。AMCL 通过自适应粒子数管理和绑架恢复机制，在保持 MCL 优势的同时，解决了效率与鲁棒性问题，从而成为 ROS 导航栈中的核心定位模块。

amcl 功能包包含在 ROS 的导航栈（Navigation Stack）中，因此可以通过安装整个导航功能包来获取它，也可以单独安装。安装命令如下：

```
sudo apt install ros-<你的 ROS 版本>-navigation //安装整个导航包
sudo apt install ros-<你的 ROS 版本>-amcl //单独安装 amcl 包
```

amcl 的源码是开源的，可以下载并使用，从 GitHub 官方仓库下载的命令如下：

```
git clone https://github.com/ros-planning/navigation.git
```

克隆完成后，进入 navigation 目录，AMCL 的源码便位于 amcl/子目录中。

amcl 功能包的核心节点是 amcl，该节点订阅的话题、发布的话题、服务以及相关参数可查阅网址：<https://wiki.ros.org/amcl>。

amcl 功能包的使用需编写与 amcl 节点相关 launch 文件，可参考 github 的演示 launch 文件：https://github.com/ros-planning/navigation/blob/noetic-devel/amcl/examples/amcl_*.launch。

3. rf2o 激光里程计

rf2o_laser_odometry 是一个在 ROS 中广泛使用，通过 2D 激光雷达数据快速计算高精

度里程计的开源功能包。功能包的核心节点是 `rf2o_laser_odometry`，主要功能是订阅激光雷达的扫描数据，并发布机器人的里程计估计，相关节点订阅发布的话题以及相关参数可查阅网址：https://wiki.ros.org/rf2o_laser_odometry。

`rf2o_laser_odometry` 功能包的使用需编写与 `rf2o_laser_odometry` 节点相关 launch 文件，可参考 github 的演示 launch 文件：https://github.com/MAPIRlab/rf2o_laser_odometry/blob/ros1/launch/rf2o_laser_odometry.launch。

四、实验内容

1、仿真实验

内容描述：在预先构建完成的静态地图中，实现移动机器人的全局定位和精准跟踪。

具体流程如下：

1) 添加定位功能包

将 `demo04_localization` 文件夹里面的 `amcl` 文件夹直接复制到工作空间的 `src` 文件夹下，`amcl` 是一个基于自适应蒙特卡洛算法的定位功能包。为了方便功能包的管理，可以在 `src` 文件夹下新建文件夹 `ebot_navigation`，然后将 `amcl` 功能包移入。

在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim/src
$ mkdir ebot_navigation && cd ebot_navigation
$ cp -r /path/to/demo04_localization/amcl .
```

2) 编写与 `amcl` 节点相关的 launch 文件

将 `amcl` 功能包 `examples` 文件夹下的 `amcl_diff.launch` 文件复制到 `ebot_bringup` 功能包下的 `launch` 文件夹中，重命名 `nav01_amcl.launch`，并进行修改。

在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup/launch
$ cp ../../amcl/examples/amcl_diff.launch nav01_amcl.launch
$ gedit nav01_amcl.launch
```

修改内容示例如下：

```
<!-- 找到 odom_frame_id 参数 -->
<param name="odom_frame_id" value="odom"/>
<!-- 在下面添加如下两行内容 -->

<param name="base_frame_id" value="base_footprint"/>
<param name="global_frame_id" value="map"/>
```

3) 编写测试文件

在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup/launch
$ gedit test_amcl.launch
```

`test_amcl.launch` 文件输入内容示例如下：

```
<launch>

<!-- 设置地图的配置文件 -->
```

```

<arg name="map" default="nav.yaml" />

<!-- 运行地图服务器，并且加载设置的地图-->
<node name="map_server" pkg="map_server" type="map_server"
args="$(find ebot_bringup)/maps/$(arg map)"/>

<!-- 加载 amcl 文件，启动 AMCL 定位节点，与地图匹配定位 -->
<include file="$(find ebot_bringup)/launch/nav01_amcl.launch" />

</launch>

```

4) 编译执行

编译并加载环境变量，在终端上运行以下命令：

```

$ cd ~/catkin_ebot_sim
$ catkin_make
$ source ./devel/setup.bash

```

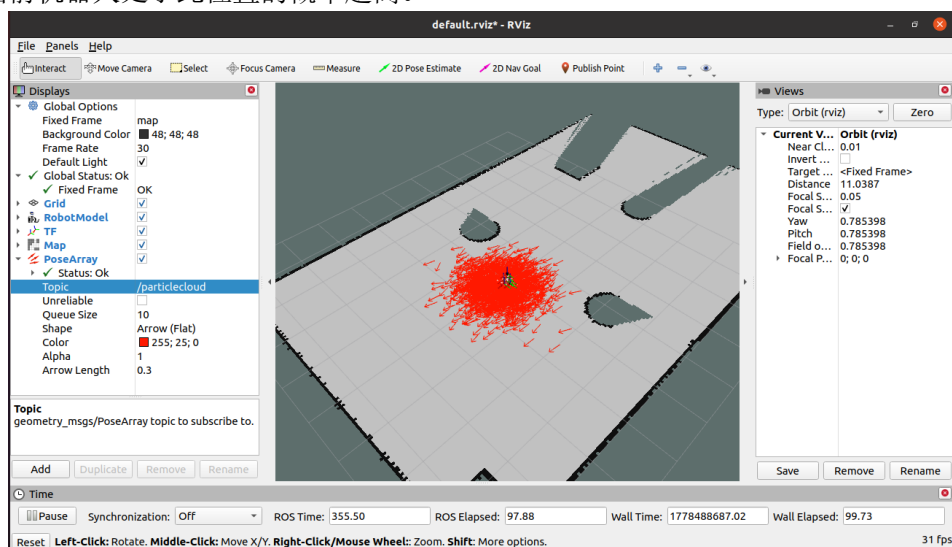
加载机器人模型和 gazebo 仿真环境，并启动 rviz，终端 1 上运行以下命令：

```
$ roslaunch ebot_bringup bringup.launch
```

启动 amcl 定位节点、地图服务，终端 2 上运行以下命令：

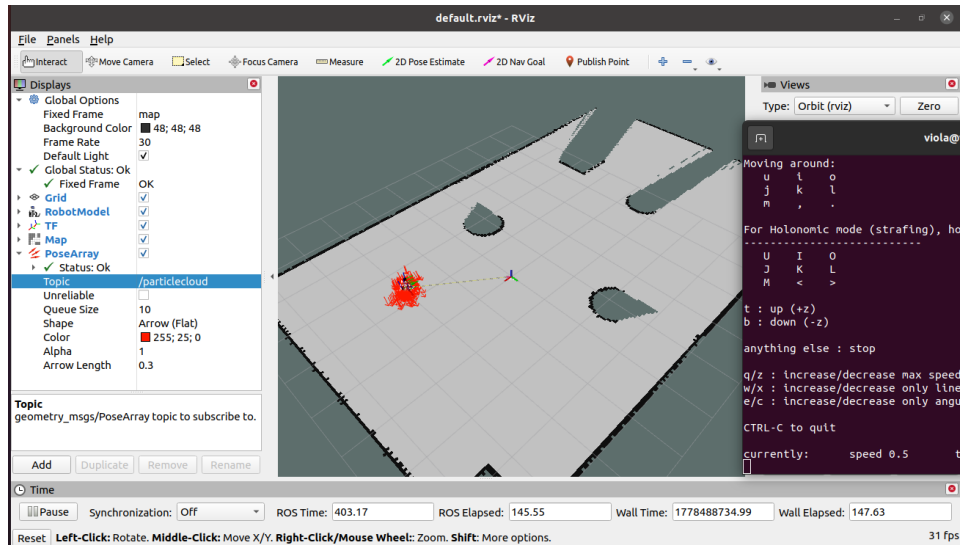
```
$ roslaunch ebot_bringup test_amcl.launch
```

在启动的 rviz 中，添加 RobotModel、TF 显示，可视化机器人模型与坐标变换关系；添加 Map 显示，设置 topic 为 /map，可视化栅格地图；添加 posearray 显示，设置 topic 为 /particlecloud，可视化定位功能包预估的当前机器人的位姿，箭头越是密集，说明当前机器人处于此位置的概率越高。



启动键盘遥控节点，终端 3 上运行以下命令：

```
$ rosrn teleop_twist_keyboard teleop_twist_keyboard.py
```



通过键盘遥控机器人运动，会发现 posearray 也随之而改变。

注：如果添加 Map 显示后，地图加载出现问题，查看 yaml（如 nav.yaml）文件里地图图片的路径，将该路径设置为地图图片（如 nav.pgm）的绝对路径，或者相对于.yaml（如 nav.yaml）文件的相对路径。

2、实车实验

内容描述：在预先构建完成的静态地图中，实现移动机器人的全局定位和精准跟踪。

具体流程如下：

1) 添加功能包

将 demo04_localization 文件夹里面的 amcl 和 rf2o_laser_odometry 两个文件夹直接复制到工作空间的 src 文件夹下。为了方便功能包的管理，可以在 src 文件夹下新建文件夹 ebot_navigation，然后将两个功能包移入。

在终端上运行以下命令：

```
$ cd ~/test_demo/catkin_ebot/src
$ mkdir ebot_navigation && cd ebot_navigation
$ cp -r /path/to/demo04_localization/rf2o_laser_odometry .
$ cp -r /path/to/demo04_localization/amcl .
```

2) 编写与 amcl 节点相关的 launch 文件

将 amcl 功能包 examples 文件夹下的 amcl_diff.launch 文件复制到 ebot_bringup 功能包下的 launch 文件夹中，重命名为 nav01_amcl.launch，并进行修改。

修改内容示例如下：

```
<!-- 找到 odom_frame_id 参数 -->
<param name="odom_frame_id" value="odom"/>
<!-- 在下面添加如下两行内容 -->

<param name="base_frame_id" value="base_footprint"/>
<param name="global_frame_id" value="map"/>
```

3) 编写与 rf2o_laser_odometry 节点相关的 launch 文件

将 rf2o_laser_odometry 功能包 launch 文件夹下的 rf2o_laser_odometry.launch 文件复制到 ebot_bringup 功能包下的 launch 文件夹中, 重命名 nav01_rf2o.launch, 并进行修改。

nav01_rf2o.launch 文件输入内容示例以下:

```
<launch>
  <node pkg="rf2o_laser_odometry" type="rf2o_laser_odometry_node"
name="rf2o_laser_odometry" output="screen">
    <param name="laser_scan_topic" value="/scan"/>
    <param name="odom_topic" value="/rf2o_laser_odometry/odom" />
    <param name="publish_tf" value="true" />
    <param name="base_frame_id" value="base_footprint"/>
    <param name="odom_frame_id" value="odom" />
    <param name="init_pose_from_topic" value="" />
    <param name="freq" value="6.0"/>
    <param name="verbose" value="true" />
  </node>
</launch>
```

4) 编写测试文件

编写用于测试定位功能的 launch 文件 test_amcl.launch, 在终端上运行以下命令:

```
$ cd ~/test_demo/catkin_ebot/src/ebot_bringup/launch
$ gedit test_amcl.launch
```

test_amcl.launch 文件输入内容示例如下:

```
<launch>
  <!-- 设置地图的配置文件 -->
  <arg name="map" default="nav.yaml" />

  <!-- 运行地图服务器, 并且加载设置的地图 -->
  <node name="map_server" pkg="map_server" type="map_server"
args="$(find ebot_bringup)/maps/$(arg map)"/>

  <!-- 启动 rf2o_laser_odometry 节点, 发布里程计 -->
  <include file="$(find ebot_bringup)/launch/nav01_rf2o.launch"/>

  <!-- 加载 amcl 文件, 启动 AMCL 定位节点, 与地图匹配定位-->
  <include file="$(find ebot_bringup)/launch/nav01_amcl.launch" />

</launch>
```

5) 编译执行

编译并加载环境变量, 在容器终端上运行以下命令:

```
$ cd ~/test_demo/catkin_ebot
$ catkin_make
$ source ./devel/setup.bash
```

启动小车底盘和激光雷达，在容器的第 1 个终端上运行以下命令：

```
$ roslaunch ebote_bringup bringup.launch
```

启动里程计发布和定位节点，在容器的第 2 个终端上运行以下命令：

```
$ roslaunch ebote_bringup test_amcl.launch
```

6) 键控机器人构建地图

启动 rviz，在远程 PC 机的第 1 个终端上运行以下命令：

```
$ rviz
```

在启动的 rviz 中，添加 TF、LaserScan 显示，可视化坐标变换关系与雷达扫描点；添加 Map 显示，设置 topic 为/map，可视化栅格地图；添加 posearray 显示，设置 topic 为/particlecloud，可视化定位功能包预估的当前机器人的位姿，箭头越是密集，说明当前机器人处于此位置的概率越高。

启动键盘遥控节点，在远程 PC 机的第 2 个终端上运行以下命令：

```
$ roslaunch teleop_twist_keyboard teleop_twist_keyboard.py
```

通过键盘遥控机器人运动，会发现 posearray 也随之而改变。

五、实验要求

1、基础实验

在预先构建完成的静态地图中，实现移动机器人的全局重定位和精准跟踪。通过 RViz 观察机器人位姿、粒子云分布、坐标变换关系和定位跟踪效果。

更换不同分辨率的静态地图，对比定位跟踪效果。

2、AMCL 参数调优实验

分别调整 amcl 功能包中的参数，对比调整前后的定位效果。例如：粒子数量（min_particles、max_particles）、更新频率（update_min_d、update_min_a）、重采样噪声（odom_model_type、alpha 参数）、重定位阈值（recovery_alpha_slow、recovery_alpha_fast）等。

3、抗干扰与鲁棒性实验

针对人为短暂遮挡激光雷达、推动机器人产生位姿（机器人绑架问题）等突变情况，观察定位是否丢失、能否自动恢复。

4、思考问题

1) 请分析 map、odom、base_footprint 三个坐标系的含义，以及坐标变换关系与数据传递逻辑。

2) 导航系统的 TF 变换树为什么必须设计成 map -> odom -> base_footprint 这样的链式结构？为什么定位算法（如 AMCL）不直接发布 map -> base_footprint 的变换？

3) 不同的定位与建图算法，分别负责发布 TF 树中的哪一段变换？例如，轮式里程计、AMCL 以及 SLAM 算法（如 Gmapping、Hector）各自对应哪一段 TF？

4) rf2o_laser_odometry 和 gmapping 功能包结合, 实体机器人可以完成地图构建吗? 如果可以, 请给出操作步骤; 如果不可以, 请给出原因。

六、实验报告要求

1. 提交时间: 两周之内提交
2. 页数: 4~6 页以内
3. 报告内容: 实验目的、实验步骤与结果分析、结论与问题讨论
4. 提交方式: 电子版和纸质版均需提交;

电子版: 实验报告和代码打包, 加密压缩后, 上传至网盘 <http://58.206.101.71:8266/>;

压缩包名: 班级+学号+姓名, 报告格式为 word 或 pdf;

纸质版: 附上电子版密码, 黑白双面打印即可, 由各班学委收齐后交至科学馆 303。

七、参考资料

1. 张新钰, 赵虚左, 丘楠, 郭世纯, 《ROS 机器人理论与实践》, 清华大学出版社
2. 课程资料:

<http://www.autolabor.com.cn/book/ROSTutorials/>

3. 课程讲解 (B 站):

https://www.bilibili.com/video/BV1Ci4y1L7ZZ/?spm_id_from=333.999.0.0&vd_source=4913ab2f45996351f46d15faddba2bfd

注: 本次实验主要涉及《ROS 机器人理论与实践》第七章内容, 重点参阅 7.2.3 内容。